# HW1: Mid-term assignment report

*Inês Simões Batista [124877]*, v2026-10-29


**1 Introduction** **2**
1.1 Overview of the work 2
1.2 Current implementation (faults & extras) 3
1.3 Use of generative AI 4

**2 Product specification** 4
2.1 Functional scope and actors 4
2.2 Parcel lifecycle 7
2.3 Business rules 10
2.4 Non-functional requirements 13
2.5 System architecture and design decisions 14
2.6 API for developers 18

**3 Quality assurance** 19
3.1 Overview of the tests available 19
3.2 Unit tests and test doubles 20
3.3 Integration tests 21
3.4 Functional tests (web automation) 22
3.5 Acceptance tests (BDD) 23
3.6 Non-functional testing 23
3.8 Code quality analysis 24
3.9 Continuous integration pipeline 24

**4 References & resources** 25


## 1 Introduction

### 1.1 Overview of the work

This report presents the midterm individual project required for TQS, covering both the software product features and the adopted quality assurance strategy. TuttiBox is a small parcel-locker management system for three seeded stations in Aveiro. A partner registers a parcel through a REST API and the service reserves a compatible locker compartment. A

courier uses a web terminal to open that assigned compartment, place the parcel and confirm that the door is closed. The parcel then becomes ready for pickup and the recipient receives a one-time pickup code through a notification port. The recipient enters that code in another web terminal, the service validates it, opens the locker and releases the compartment. A tracking page exposes the current parcel state and its timestamped transition history, while an operator page filters parcels, lists expired parcels, removes an expired parcel and takes a free compartment out of service or returns it to service.

The application is organized around the parcel's full journey rather than around a single screen. The recipient starts by entering an address on the search page, the backend resolves that address to coordinates and ranks the seeded stations by geographic distance, while also reporting free capacity by compartment size. A partner system then submits a parcel-registration request directly to the REST API. The service validates the requested size, finds a compatible station compartment, reserves it and returns a tracking identifier. That identifier connects the partner's registration to the courier's drop-off, the recipient's tracking view and the operator's later supervision. The partner channel is deliberately an API because UC2 defines a system-to-system interaction meaning no partner-facing web form is part of the required product.

The courier interaction is split into opening and confirmation so that the system does not treat an opened door as proof of a completed deposit. The courier terminal sends the parcel's assigned station and compartment together with a courier identifier. Only after the fake hardware confirms that the door is closed does the service advance the parcel to READY_FOR_PICKUP, mark the compartment occupied and issue a pickup code through the notification port. The recipient terminal accepts that code without asking the recipient to identify the parcel separately. A successful pickup opens the assigned locker, advances the parcel to PICKED_UP, clears the stored code hash and frees the compartment. In parallel, the tracking page makes the current state and transition history visible, and the operator page handles exceptional inventory work such as expired parcels and compartments that should temporarily be unavailable.

The lifecycle has two deadline paths with different physical consequences. If a CREATED parcel is not deposited before its drop-off deadline, the system cancels it and releases the reservation because the locker never became occupied. If a parcel is already READY_FOR_PICKUP when its pickup deadline passes, it becomes EXPIRED but remains in an occupied compartment meaning that an operator must remove it before the compartment can become free.

The project is deliberately a testable demonstration rather than a production locker integration. PostgreSQL stores stations, compartments, parcels and their state transitions, while Spring Boot exposes REST endpoints and serves five static HTML pages. The browser layer uses HTML, CSS and JavaScript fetch calls meaning there is no separate JavaScript application build. Hardware access is behind a HardwarePort, currently backed by an in-memory fake that tracks door state and can simulate failures. Notification delivery is behind a NotificationPort, its demo adapter records the issued code for tests and writes it to application logs. Geocoding is also behind a port: a Nominatim adapter serves normal configuration, while tests and offline demonstrations can select a deterministic in-memory provider. These boundaries allow the system's business rules to be exercised without physical lockers, a messaging account or a live geocoding dependency.

For example, a complete demonstration can search for “Aveiro”, register an M parcel at ST-AVEIRO-CENTRO, copy the generated tracking code into the courier page, confirm the locker closure, retrieve the demo pickup code from the log and complete pickup in the recipient page. At every step, the identifiers returned by one operation feed the next: the assigned compartment is not chosen by the courier, and the pickup code is not available at registration time. The operator can then inspect the resulting parcel history and compartment inventory. Each visible action must correspond to a guarded state transition and a consistent update to the database, hardware fake and capacity view.

### 1.2 My current implementation (limitations & extras)

All six required use cases are represented in the implementation. UC2 is intentionally an API workflow: the assignment specifies the partner channel as REST API, so there is no partner registration page. The recipient-facing search and tracking pages, courier and recipient locker terminals, and operator page are present.

The following limitations remain material to the requirements and must not be mistaken for completed behavior:

- Pickup-code guessing protection is missing. Codes are generated securely and stored as hashes, but there is no per-code or per-client attempt limit, delay, temporary lockout or other throttling at /api/parcels/pickup so unlimited syntactically valid guesses can be submitted. The assignment explicitly asks for protection against guessing so it’s something that is missing.
- The public Nominatim policy is not fully met by the UI. The adapter identifies the application with a custom User-Agent and geocoding is user-triggered, but the application does not enforce Nominatim's maximum one request per second across users and does not display OpenStreetMap attribution in the search page. The limit=1 request parameter limits returned search results, it does not rate-limit requests. There is no geocoding cache.
- Accessibility verification is partial. Markup-level usability tests check labels, language, viewport, landmarks, status regions and navigation.
- Deadline calculations use an injected Clock, but transition timestamps use LocalDateTime.now() directly in ParcelTransition. Deadline rules can be tested without waiting, but transition timestamp tests are not fully controlled by the same clock.
- The Testcontainers PostgreSQL image uses postgres:latest, while the local Compose service uses PostgreSQL 16. Both are PostgreSQL, as required, but pinning the test image to the same major version would make the environment more reproducible.

The project also contains Playwright browser tests, a GitHub Actions workflow, JaCoCo coverage enforcement and an optional local SonarQube Compose service.

### 1.3 Use of generative AI

AI was used as a supporting tool in the development of this project. Initially, it helped me understand the state machine, I had only a very vague idea of how it would eventually be implemented, and it was the AI that made me realize the system was not as simple as it first

appeared. I used AI specifically to work on the logic for the "fake hardware" and the geocoding functionality. I was able to implement the lifecycle of an parcel, along with the associated tests, largely on my own.

I also used AI to help develop the interface and the vast majority of the usability tests, while I was able to conceive the tests and define what I wanted to evaluate, I still struggle with the syntax and the actual coding process.

One specific decision I changed after running my code through an AI concerned the state-transition logic for an order: I shifted it so that the state only changes when the physical hardware door is opened or closed. In my initial implementation, the state would change as soon as the validation code was entered, I hadn't initially considered the possibility of hardware failures or the chance that the entered code might not actually correspond to the order. This is an important check, and it was only implemented thanks to a suggestion from the AI.

The AI I used to assist with coding, and to help me think through the logic that still needed to be implemented to make the system robust and thoroughly tested was Big Pickle. Whenever this AI generated code, I would review the "diff" file it provided to see exactly what had been changed or added. I made a point of understanding what it was doing, if I didn't grasp something, I would ask for clarification. Furthermore, if the code it produced didn't meet my standards, I would keep it in the project but commented outthat way, I had a reference that clearly described the intended functionality, even if the implementation itself was poor, allowing me to write a much more suitable version.

# 2 Product specification

## 2.1 Functional scope and actors

The implementation covers the six required use cases. Automated criteria below distinguishes Cucumber scenarios from browser automation: Cucumber drives the HTTP/API layer through Spring MockMvc and Playwright drives the actual web pages in Chromium.

| ID | User story | Acceptance criteria | Acceptance test |
|---|---|---|---|
| US1 | As a recipient, I want to enter an address and find nearby lockers so that I can choose a station with suitable capacity | The address is geocoded so stations are sorted by distance and each result shows the station code/name, distance and free S/M/L capacity. An unknown address returns an empty result and provider failures are reported explicitly | ● Uc1_station_search.feature<br>● StationSearchServiceTest<br>● StationSearchControllerIntegrationTest<br>● ApiContractTest<br>● The playwright station-search journey<br>● WebInterfaceUsabilityTest |

| ID | User story | Acceptance criteria | Acceptance test |
|---|---|---|---|
| US2 | As a partner, I want to register a parcel for a selected station so that a compatible compartment is reserved and I receive a tracking code | A valid request contains station, size, recipient contact and partner ID, the smallest compatible free compartment is reserved, the response contains the generated tracking code and assignment, invalid input is rejected, a missing station returns 404, no compatible compartment returns 409, concurrent registrations cannot double-book the last compartment | • uc2_parcel_registration.feature<br>• RegisterParcelIntegrationTest<br>• ParcelControllerIntegrationTest<br>• ParcelAllocationConcurrencyIntegrationTest<br>• ApiConcurrencyIntegrationTest<br>• ApiContractTest |
| US3 | As a courier, I want to open the reserved compartment and confirm its door is closed so that the parcel can be marked ready for pickup | The tracking code, station, assigned compartment and courier ID are supplied and only the assigned, size-compatible compartment can be used, the parcel remains CREATED until door closure is confirmed and then it becomes READY_FOR_PICKUP, the compartment becomes occupied and a pickup code is issued, also hardware failures do not falsely complete the deposit | • uc3_courier_dropoff.feature<br>• CourierDropOffIntegrationTest<br>• ParcelServiceIntegrationTest<br>• PlaywrightEndToEndTest |
| US4 | As a recipient, I want to enter the pickup code so that the assigned compartment opens and I can collect the parcel | Only a correctly formatted, matching code for a READY_FOR_PICKUP parcel before its pickup deadline is accepted, a successful pickup opens the assigned compartment, changes the state to PICKED_UP, clears the stored hash and frees the compartment and an invalid, expired, reused or hardware-failed pickup is rejected without consuming a valid code | • uc4_recipient_pickup.feature<br>• RecipientPickUpIntegrationTest<br>• ParcelServiceTest<br>• ParcelLifecycleConcurrencyIntegrationTest<br>• PlaywrightEndToEndTest |

| ID | User story | Acceptance criteria | Acceptance test |
|---|---|---|---|
| US5 | As a recipient or partner, I want to check a tracking code and see the current state and history so that I can follow the parcel lifecycle | The web page displays the current state and ordered transition entries, REST offers parcel details, status and history, an unknown tracking code returns 404 and each transition includes from/to states, actor and timestamp | • uc5_tracking_and_lifecycle.feature<br>• ParcelHistoryIntegrationTest<br>• TrackingPageIntegrationTest<br>• ApiContractTest<br>• Playwright lifecycle journey |
| US6 | As an operator, I want to supervise a station so that I can inspect parcels, remove expired parcels and manage faulty compartments | Parcels can be filtered by station and state, expired parcels are listed, removing one transitions it to RETURNED and frees its compartment, a free compartment can be put out of service and restored and an occupied/reserved compartment cannot be taken out of service | • uc6_operator_supervision.feature<br>• OperatorSupervisionIntegrationTest<br>• OperatorTerminalPageIntegrationTest<br>• Playwright compartment-management journey |

The seed and the external geocoding API provide different information. SeedData creates TuttiBox's own station and compartment inventory in PostgreSQL: the three station records, their coordinates and the 18 lockers whose availability the application manages. Nominatim does not return TuttiBox lockers or their capacity. It only translates the address entered by the recipient into a latitude/longitude point. The search service compares that point with the seeded (and subsequently database-managed) station coordinates, calculates distances, and joins the results with the current free-compartment counts. Without station records in the database, a successful Nominatim response would still produce no TuttiBox results because the application would have no network locations to rank.

## 2.2 Parcel lifecycle

Parcel is the aggregate that owns its lifecycle transitions. A new parcel starts at CREATED, the initial history entry records a transition from null with actor PARTNER. State-changing methods guard their allowed source state and call transitionTo, which changes the state, appends a ParcelTransition, and logs the transition.

The first transition is created by partner registration. The service validates the request, reserves one compatible compartment and stores the drop-off deadline, the parcel becomes CREATED and its initial history entry identifies PARTNER as the actor. While it remains CREATED, the courier may begin a drop-off by asking the hardware port to open that assigned compartment, but this opening alone does not change the parcel state. The courier must confirm the door closure, and the hardware port must both accept the close confirmation and report a closed door. Only then does the service move the parcel to

READY_FOR_PICKUP, mark the compartment OCCUPIED, set the pickup deadline, persist the hash of a newly generated pickup code and send the plaintext code through the notification port.

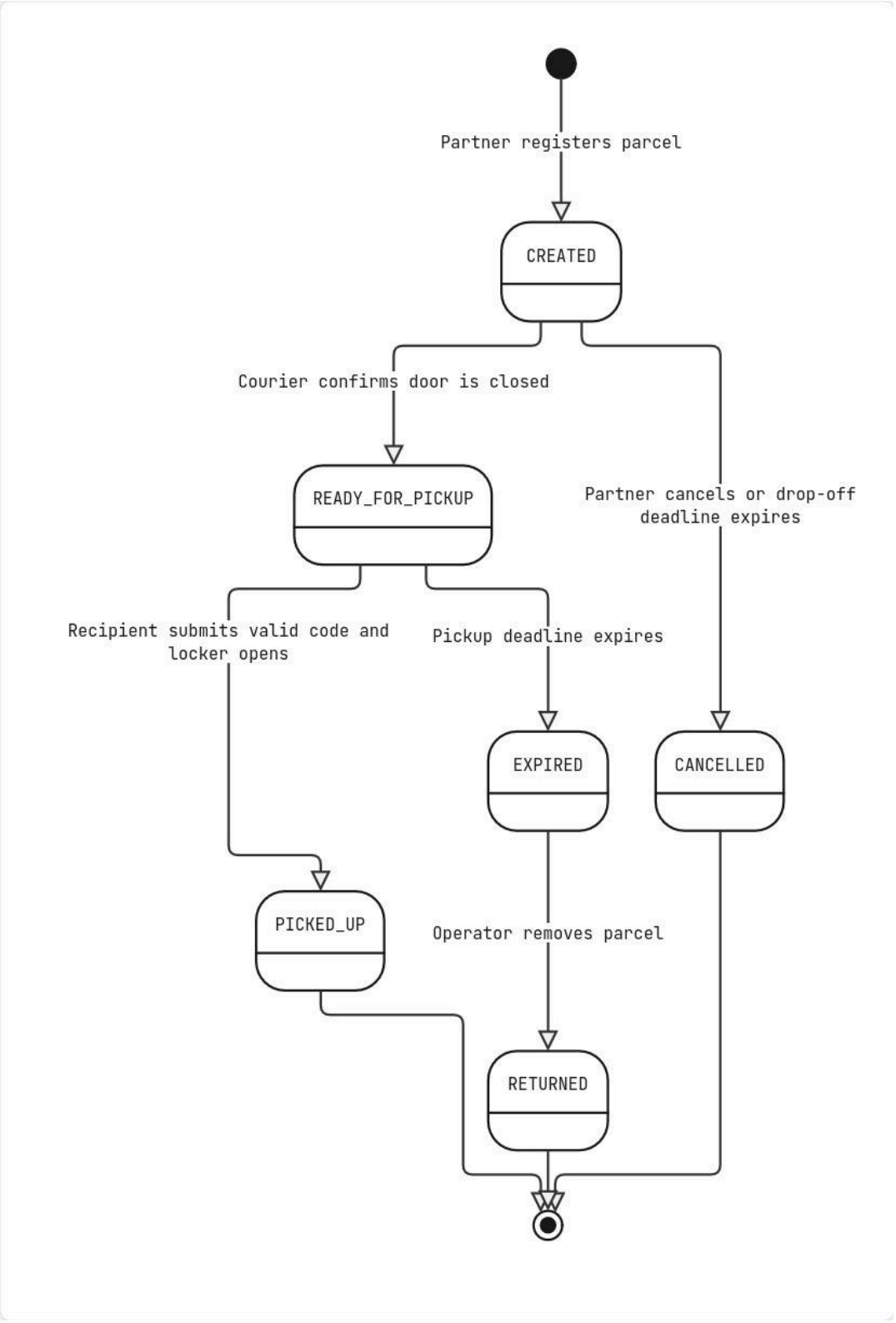
A CREATED parcel can instead become CANCELLED if the partner cancels it or if its drop-off deadline expires before deposit. In either case the reservation is released and the compartment becomes FREE. From READY_FOR_PICKUP, the recipient can submit a valid, unused pickup code before the pickup deadline, after the assigned locker opens, the state becomes PICKED_UP, the stored code hash is cleared and the compartment becomes FREE. If the pickup deadline passes first, the parcel becomes EXPIRE, but the compartment remains OCCUPIED because the parcel is still physically inside. The operator must remove the expired parcel to move it to RETURNED and release the compartment. CANCELLED, PICKED_UP and RETURNED are terminal parcel states in the current model.

These transition guards are observable at the API boundary. Invalid lifecycle transitions are mapped to HTTP 400, and missing parcels to 404. A pickup attempted after its deadline is rejected immediately, the scheduler changes the parcel to EXPIRED on its next run, up to the configured scheduler interval later. Thus an unsuccessful pickup attempt does not itself expire the parcel, and an EXPIRED parcel never implies an empty compartment. The distinction is reflected in persisted state and is important to both the recipient's tracking view and operator inventory.

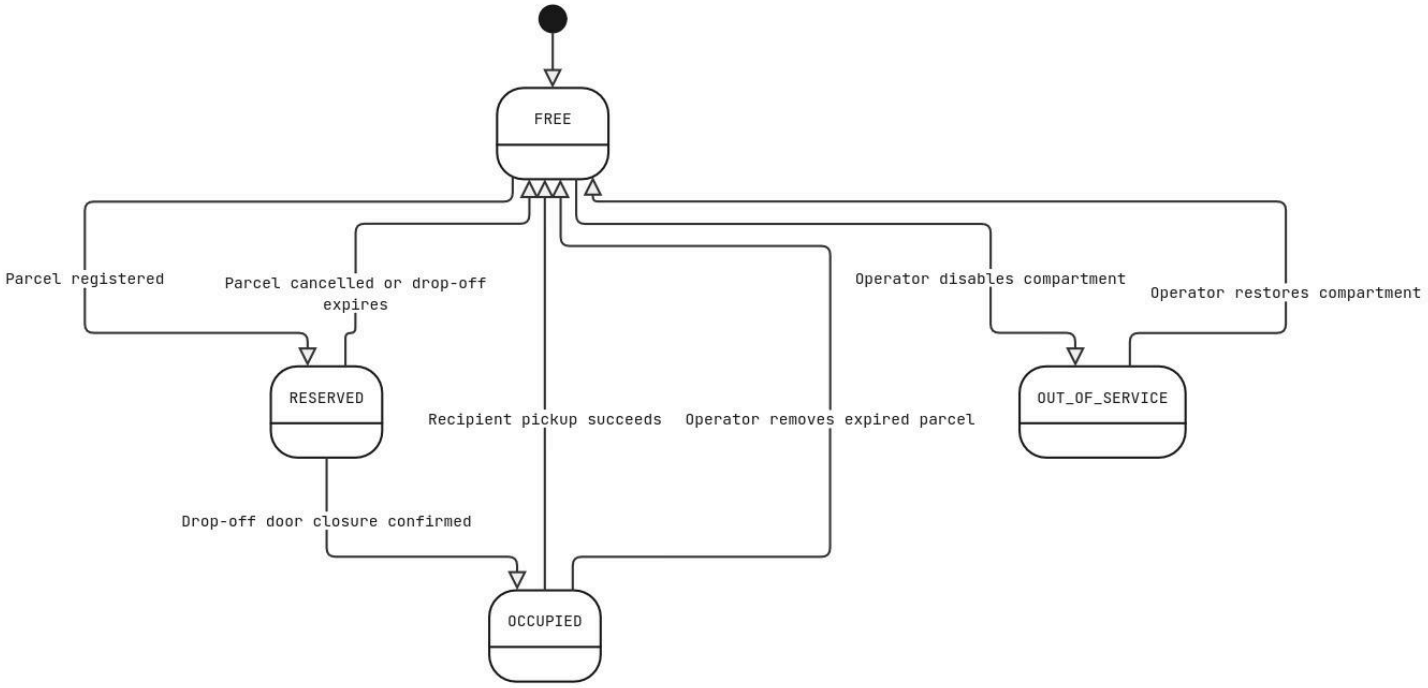
Operations that do not satisfy a transition's guard fail explicitly rather than changing state. Invalid lifecycle transitions are mapped to HTTP 400 and missing parcels to 404. A pickup attempted after its deadline is rejected immediately, and the scheduler changes the state to EXPIRED on its next run (up to the configured scheduler interval later).

For example, after successful registration and deposit, the parcel contains at least these two entries: (null -> CREATED, actor=PARTNER) and (CREATED -> READY_FOR_PICKUP, actor=COURIER-DEMO). A successful pickup adds (READY_FOR_PICKUP -> PICKED_UP, actor=RECIPIENT). The timestamp lives on each ParcelTransition so the tracking page renders the entries in order as a timeline, while GET /api/parcels/{trackingCode}/history exposes the records as JSON. The tests check both the resulting current state and selected history rows.

The parcel and compartment state machines are related but not identical. For example, EXPIRED is not an empty locker: the parcel is still physically represented by an OCCUPIED compartment until the operator removes it. The transition to RETURNED and the compartment transition to FREE occur together in the operator removal operation. Likewise, an invalid pickup code is not a state transition at all: state, hash and compartment status must remain unchanged.




Compartments have an independent state machine. A parcel registration takes the smallest compatible FREE compartment, a completed courier drop-off makes it OCCUPIED, cancellation, pickup or operator removal frees it. Expired parcels keep the compartment occupied until removal. Only FREE compartments may be taken out of service.




### 2.3 Business rules

| Rule/theme | Value/rationale | Configured in | Tested in |
|---|---|---|---|
| Compartment allocation | Sizes are ordered S < M < L, an S parcel fits any size, M fits M/L and L fits only L. The query orders by size and then code, preserving large compartments where a smaller one is available. One parcel can reserve a compartment at a time | The size ordering is domain code, stations and compartment sizes are seed data in SeedData | • CompartmentSizeTest<br>• CompartmentTest<br>• RegisterParcelIntegrationTest<br>• UC2 and allocation concurrency tests |
| Full station / no compatible compartment | Registration is rejected with HTTP 409 and NoAvailableCompartmentException, no parcel is created if allocation fails | No separate property, it is determined from persisted free compartments | • RegisterParcelIntegrationTest<br>• uc2_parcel_registration.feature |
| Concurrent allocation | Allocation runs inside a transaction and locks candidate compartments with JPA PESSIMISTIC_WRITE, the test starts concurrent requests against the last available compartment | Locking/query is implemented in CompartmentRepository.findAvailableByStationAndSizes | • ParcelAllocationConcurrencyIntegrationTest<br>• API concurrency test and UC2 concurrent scenario |

| Rule/theme | Value/rationale | Configured in | Tested in |
|---|---|---|---|
| Drof-off deadline | 48 hours after registration, an overdue CREATED parcel is cancelled and its compartment released. Drop-off is also guarded when a courier requests it after the deadline | tuttibox.deadlines.drop-off=PT48H and DeadlineProperties.dropOff | • ParcelTest<br>• ParcelDeadlineIntegrationTest<br>• ParcelServiceTest<br>• uc7_deadlines.feature |
| Deadline scheduler | Enabled by default and runs after a fixed delay of 60,000 ms and scheduling can be disabled for tests | tuttibox.deadlines.check-interval-ms=60000 and tuttibox.deadlines.scheduling-enabled=true | • ParcelLifecycleSchedulerTest and scheduler scenario in uc7_deadlines.feature<br>• integration test properties disable background scheduling |
| Pickup-code generation and format | Eight characters selected using SecureRandom from ABCDEFGHJKLMNPQRSTUVWXYZ23456789 (32 symbols, 40 bits) with ambiguous I,O,0 and 1 are omitted to reduce entry mistakes. Format is case-insensitive after normalization | Alphabet and length are constants in PickupCode | • PickupCodeTest<br>• RecipientPickUpIntegrationTest<br>• uc4_recipient_pickup.feature |
| Pickup-code storage and single use | SHA-256 hash is persisted meaning plaintext is delivered only to the notification adapter. Successful pickup nulls the stored hash, so the same code cannot be used again. Constant-time digest comparison uses MessageDigest.isEqual | Algorithm and format are constants in PickupCode abnd code hash is a unique 64-character database column | • PickupCodeTest<br>• ApiContractTest verifies no hash is returned, pickup integration and concurrency tests |
| Pickup-code validity and guessing | Valid only for a parcel in READY_FOR_PICKUP and before its seven-day pickup deadline | Deadline configurable as above | • State/deadline and invalid-code scenarios exist<br>• There is no test proving a guessing limit because no limit is implemented |

| Rule/theme | Value/rationale | Configured in | Tested in |
|---|---|---|---|
| Hardware: open failure | Drop-off does not move from CREATED and the reserved compartment remains reserved if hardware cannot open. During pickup, a failed open leaves the parcel ready, code usable and compartment occupied | Failure injection in InMemoryHardwareFake | • ParcelServiceTest<br>• ParcelServiceIntegrationTest<br>• UC3/UC4 hardware-failure scenarios |
| Hardware: close confirmation failure | The parcel is not marked READY_FOR_PICKUP and the compartment is not marked OCCUPIED if the close operation throws | setInjectCloseFailure(true) on the fake | • ParcelServiceTest<br>• ParcelServiceIntegrationTest<br>• CourierDropOffIntegrationTest |
| Hardware: door remains open | Even if the close call returns, the service checks isDoorClosed and a false reading prevents the state transition and code issue | setInjectDoorNotClosed(true) on the fake | • ParcelServiceIntegrationTest<br>• CourierDropOffIntegrationTest |
| Geocoding | Search uses public Nominatim /search, JSONv2, a result limit of 1, Portuguese country and language filters, a custom application User-Agent and a five-second timeout. Unknown results produce an empty station list, provider problems map to explicit errors | tuttibox.geocoding.* properties in application.properties and GeocodingPropertiesmeaning provider can be switched between nominatim and memory | • Nominatim adapter tests against a local JDK HttpServer, geocoding fake tests, station-search/controller tests |
| Geocoding provider policy | Requests are user-triggered and identified, but there is no one-request-per-second throttle, no cache and no visible OSM attribution. Therefore public Nominatim policy compliance is incomplete | No request-rate or attribution setting exists | • Not verified |

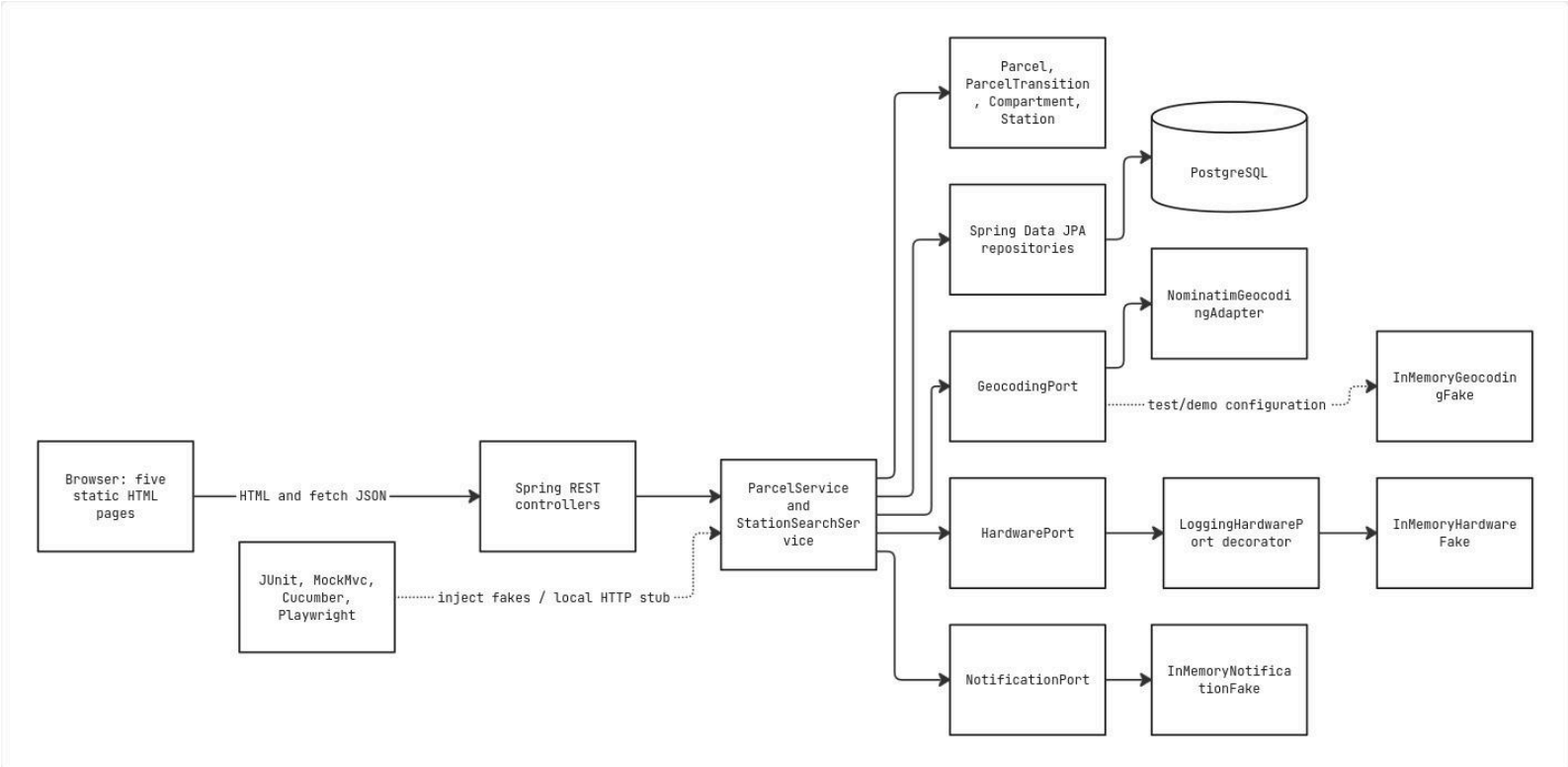
### 2.4 Non-functional requirements

| ID | Requirement (measurable) | Verification method/tests implemented |
|---|---|---|
| NFR1 | Compile for Java 21 and persist application data in a relational database | Maven build uses Java 21, local application Compose uses PostgreSQL 16 and test integration suites use PostgreSQL Testcontainers. Last available local Surefire reports show the complete suite passed. Test container tag is currently postgres:latest, so version parity is not pinned |
| NFR2 | Under two simultaneous registrations for the final compatible compartment, at most one registration succeeds and no compartment is double-booked | Parallel integration tests coordinate worker threads using latches and verify results and database state. This verifies a concurrency invariant, not load capacity |
| NFR3 | Invalid transitions, validation, missing entities, full stations, hardware failures and geocoding failures return an explicit status and JSON body | GlobalExceptionHandlerIntegrationTest, controller suites, Cucumber and JSON Schema contract tests. The common body includes timestamp, numeric status, reason phrase and message, validation returns the first field error |
| NFR4 | A geocoding request times out within the configured five-second HTTP timeout, 429, 5XX, malformed JSON, empty result and connection failure have defined outcomes | Adapter tests use a local stub HTTP server and do not call real Nominatim, controller tests verify HTTP mapping. This does not verify Nominatim's service-level availability |
| NFR5 | Do not send confidential user data, identify the app, limit rate to at most one request/second, avoid API autocomplete, and show OpenStreetMap attribution | The custom User-Agent is configured, the UI's datalist is static and tests never call the real API. Rate throttling and visible attribution are missing. |
| NFR6 | All five pages declare Portuguese, UTF-8 and viewport metadata and expose a main landmark, labels and status feedback where applicable | WebInterfaceUsabilityTest checks rendered HTML source, and Playwright verifies the five pages rendered in Chromium. This is a limited automated check, not evidence. |
| NFR7 | Define and meet a response-time/throughput objective under a stated concurrent user/load profile | hw1/performance/k6/tuttibox.js now supplies an exploratory 55-second mixed workload, station searches peaking at five iterations/second and static-page requests peaking at two/second. The local run met its script thresholds (global p95 10.38 ms, 0 failed HTTP requests, 0 dropped iterations) |

| ID | Requirement (measurable) | Verification method/tests implemented |
|---|---|---|
| NFR8 | Code has high enough entropy, is not stored in plaintext, can only be used once and cannot be guessed through unlimited API attempts | SecureRandom, 40-bit format, SHA-256 hash storage, constant-time comparison and single-use state behavior are implemented and tested. Unlimited attempts remain possible, so the anti-guessing criterion is unmet |
| NFR9 | The full build maintains at least 90% line and 80% branch coverage | JaCoCo verify gate enforces both. The available jacoco.xml reports 97.09% line coverage (701/722) and 86.93% branch coverage (153/176), above the configured thresholds |

### 2.5 System architecture and design decisions

The application follows a layered structure with explicit ports for geocoding, locker hardware and notifications. Spring Data repositories provide the persistence-facing boundary. The adapters are replaceable in tests; the production/demo configuration intentionally uses simulated hardware and a log-backed notification fake.




The backend uses Java 21, Spring Boot 4.1.1, Spring MVC, Spring Data JPA, Hibernate, Bean Validation and PostgreSQL. Maven Wrapper provides repeatable build commands. JUnit 5, Mockito, AssertJ, Spring Boot Test, Testcontainers, Cucumber, JSON Schema validation and Playwright are test dependencies. Static pages under src/main/resources/static use browser fetch to call the API. Java HttpClient is used by the Nominatim adapter.

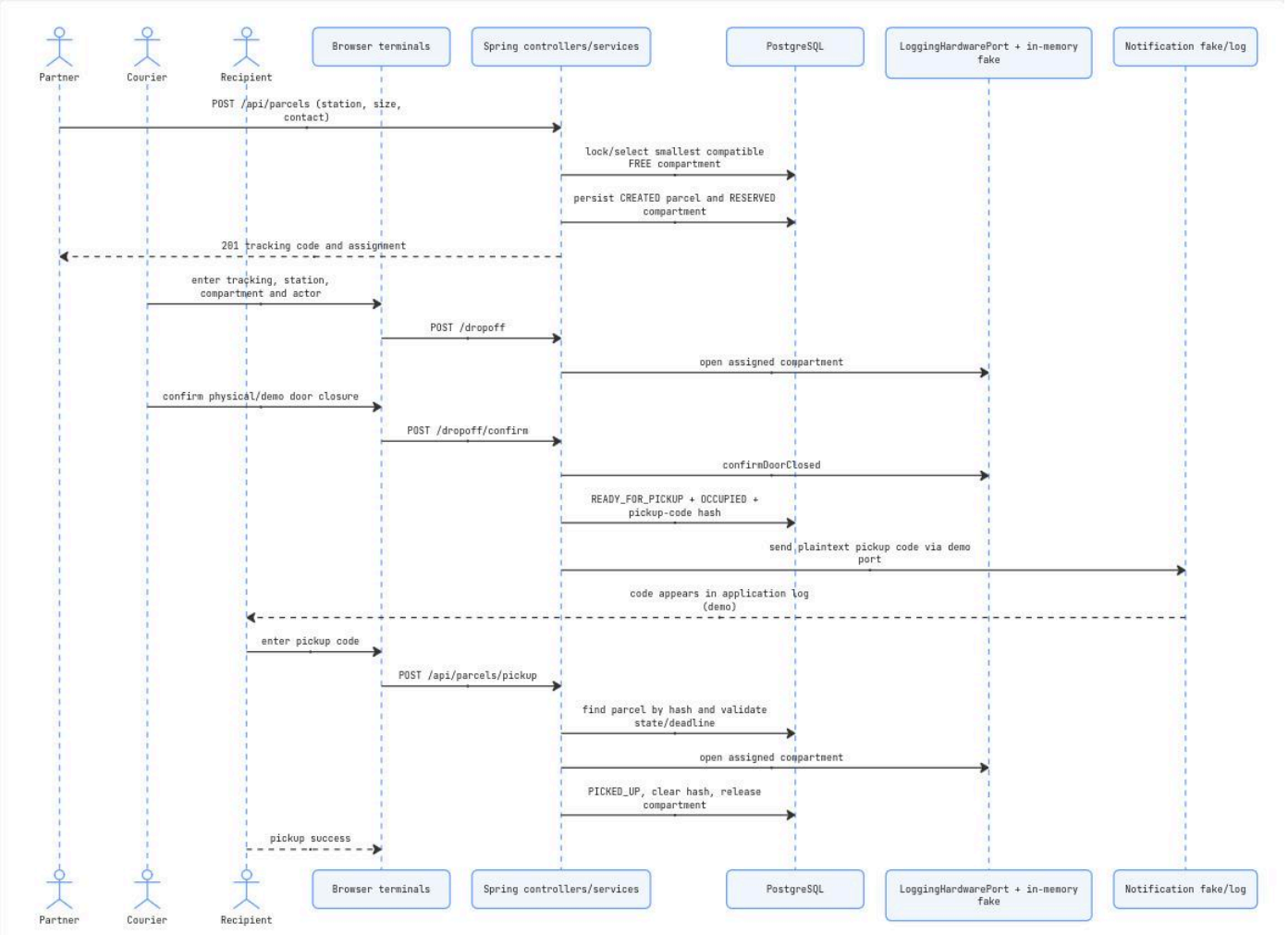
The main responsibilities are intentionally separated by package. controller translates HTTP input into service calls and return values, it should not choose a compartment or mutate lifecycle state. service/ParcelService owns the transactional use-case orchestration: find station, select/reserve a compartment, persist parcel data, call the locker port and emit notification. model/Parcel and model/Compartment reject invalid transitions even when called without an HTTP request. repository expresses persistence queries, including the locked allocation query and status/deadline lookups. geocoding contains the Nominatim-specific HTTP client while the service depends on GeocodingPort. This means the distance/ranking policy can be tested with coordinates and mocked repositories without Nominatim, while adapter tests focus on HTTP and JSON interpretation.

The implementation has one important distributed-systems boundary: the hardware call is synchronous and occurs within the service's database transaction, but a real door action cannot be rolled back if a later database operation fails. The in-memory fake and current tests exercise failures returned by the port before business-state changes meaning they do not simulate a process crash after a physical door opens or a database outage after the operation. That trade-off is acceptable for the assignment's synchronous fake, but a real locker integration would need reconciliation/idempotency or an explicit command/result protocol.

| Design decision | Your option | Testing strategy/impact |
|---|---|---|
| Representation of the parcel lifecycle | The JPA Parcel aggregate owns guarded methods for cancel, ready, pickup, expire and return, each valid transition appends a ParcelTransition. A separate state-machine library was not needed for this small fixed state set | Unit tests exercise valid and invalid transitions, service/API integration, Cucumber and concurrent endpoint tests exercise persisted behavior and error mapping |
| Controllable time in tests | ParcelService receives a Clock, the default bean is Clock.systemDefaultZone(). Drop-off and pickup durations and scheduler delay are configuration properties | Service unit tests use a fixed clock and compare before/after-boundary instants without sleeping, the scheduler is disabled in tests and invoked directly. Transition entities still call system time directly, which is a remaining determinism limitation |
| Hardware port: operations; synchronous or event-driven | Synchronous HardwarePort exposes openCompartment, confirmDoorClosed and isDoorClosed. LoggingHardwarePort decorates the in-memory implementation and logs every call/result. Event-driven messaging and real hardware protocols are out of scope | The fake tracks door state and can inject open failure, close-confirmation failure and a door that remains open, unit and service/API integration tests assert parcel/compartment state after each failure |

| Design decision | Your option | Testing strategy/impact |
|---|---|---|
| Concurrency in compartment allocation | The service transaction locks candidate free compartments using PESSIMISTIC_WRITE, orders candidates by smallest size and code, then reserves and persists the selected compartment. Optimistic retry or a separate distributed lock was not selected | PostgreSQL-backed parallel tests start requests together and inspect both HTTP results and final database state. They verify allocation safety for tested scenarios but do not measure throughput |
| Persistence port and adapter | Services depend on Spring Data repository interfaces, backed by Hibernate/JPA and PostgreSQL, a database mock is used only in isolated unit tests | Full-context integration tests use PostgreSQL Testcontainers rather than H2, preserving database-engine behavior. The test image should be version-pinned to match Compose PostgreSQL 16 |
| Pickup codes: generation, storage, checking | Eight random characters from a 32-character unambiguous alphabet are generated with SecureRandom, the DB stores SHA-256 only and normalized input is format-checked and compared with MessageDigest.isEqual so success clears the hash | Unit, API, contract, failure and concurrent pickup tests cover format, correct/incorrect code, one-use behavior, deadline and no plaintext/hash response leakage. Guess throttling is an unresolved required rule |
| Geocoding provider: choice; behavior when slow, unavailable or over limits | Public OpenStreetMap Nominatim is the default adapter meaning GeocodingPort keeps provider-specific HTTP details out of search logic. The app sends q, format=jsonv2, limit=1, countrycodes=pt, accept-language=pt, and a custom User-Agent. Timeout is five seconds. memory fake is selectable for deterministic offline demo/tests | Adapter tests run against a local JDK HttpServer stub for 429, 500, timeout, invalid JSON, malformed fields, no result and connection failure. The user-facing request path is tested with the in-memory fake. No build test calls real Nominatim. Rate throttling, caching and visible attribution remain incomplete |
| REST API design: resources, status codes, error format | Controllers use resource-oriented routes under /api/parcels, /api/stations and /api/operator, DTO validation rejects invalid input and a RestControllerAdvice maps domain exceptions to JSON | MockMvc integration tests verify success and error statuses, ApiContractTest validates representative payloads against JSON schemas |

| Design decision | Your option | Testing strategy/impact |
|---|---|---|
| Simulation of locker terminal | Five static HTML/CSS/JavaScript pages represent the recipient search, courier terminal, recipient terminal, tracking view and operator supervision. This keeps the interaction simple and avoids a client framework. The partner registration remains REST because that is the channel in UC2 | Static page availability and markup are checked with MockMvc and four Playwright E2E tests interact with a real Chromium browser. Browser tests use the local app, PostgreSQL Testcontainers and deterministic fakes |




## 2.6 API for developers

All successful requests return JSON except where the resource is a static HTML page. There is no authentication or authorization layer, the actor/courier/partner values are data used in operations and transition history, not credentials.

Station discovery is provided by GET /api/stations/search?address={address}. It geocodes the supplied address and returns an array of nearby stations with distances and available S/M/L counts. A valid address with no match returns 200 and an empty array. Missing or

invalid request input maps to 400 abd an invalid provider responses or provider errors map to 502, an unavailable or rate-limited provider maps to 503 and a timeout maps to 504.

Partners register parcels using POST /api/parcels with a JSON object containing stationId, compartmentSize (S, M or L), recipientContact and partnerId. A successful registration returns 201 Created and a ParcelResponseDto with a generated tracking code and assigned station/compartment. Validation failures return 400, an unknown station returns 404, and a station without a compatible free compartment returns 409. The parcel can then be inspected through GET /api/parcels/{trackingCode}, which returns parcel details and transition data while omitting the sensitive pickup-code hash, GET /api/parcels/{trackingCode}/status returns the current state enum value, and GET /api/parcels/{trackingCode}/history returns the transitions with from/to state, actor and timestamp. These reads return 404 when the tracking code is unknown.

Courier deposit uses two operations. POST /api/parcels/{trackingCode}/dropoff accepts stationId, compartmentId and courierId, and requests opening the parcel's assigned locker. A successful response is 200, but the parcel remains CREATED while the physical/demo deposit is in progress. Wrong assignments, invalid states and an expired drop-off deadline return 400, unknown parcels return 404, hardware failures return 503. The courier then calls POST /api/parcels/{trackingCode}/dropoff/confirm with the same fields. A successful confirmation returns 200, changes the parcel to READY_FOR_PICKUP and sends the pickup code through the notification port. The same assignment/state/deadline errors return 400, unknown parcels return 404, and hardware or door-closed failures return 503.

Recipient pickup uses POST /api/parcels/pickup with the JSON field pickupCode. On success, the API returns 200, changes the parcel to PICKED_UP, clears the stored code hash and frees the compartment. Invalid, wrong, expired or reused codes, invalid states and deadline violations return 400. An unknown code is deliberately mapped to the same generic invalid-code response rather than exposing a distinguishable 404 and hardware failure returns 503. Partner cancellation is available through POST /api/parcels/{trackingCode}/cancel?actor={actor} and applies only to CREATED parcels, success returns 200 with CANCELLED and releases the reservation. The system/test lifecycle routes POST /api/parcels/{trackingCode}/expire?actor={actor} and POST /api/parcels/{trackingCode}/return?actor={actor} respectively expire a ready parcel (200, EXPIRED) and return an expired parcel while freeing its locker (200, RETURNED). These operations return 400 for a missing actor or invalid source state and 404 for an unknown parcel.

The operator API supports inventory reads and controlled updates. GET /api/operator/parcels?stationId={id}&state={state} lists parcels with optional station/state filters, it returns 200, 404 for an unknown station and 400 for an invalid state parameter. GET /api/operator/parcels/expired?stationId={id} lists expired parcels, optionally filtered by station, and returns 404 if the requested station does not exist. POST /api/operator/parcels/{trackingCode}/remove?actor={actor} transitions an expired parcel to RETURNED and releases its compartment returning 200, 400 for a missing actor or invalid state and 404 for an unknown parcel. GET /api/operator/stations/{stationId}/compartments lists the station's compartments and returns 404 for an unknown station. The operator can disable a free compartment with POST /api/operator/stations/{stationId}/compartments/{compartmentId}/out-of-service?actor={actor} with success returns 200 and status OUT_OF_SERVICE, while a missing actor returns 400,

unknown resources return 404, and reserved/occupied compartments return 409. The corresponding .../{compartmentId}/in-service?actor={actor} action restores an out-of-service compartment to FREE with 200, missing actors return 400, unknown compartments 404 and compartments not currently out of service 409.

# 3 Quality assurance

## 3.1 Overview of the tests available

The strategy uses the narrowest test level that can observe a rule, then adds broader checks where behavior crosses component or user boundaries. Unit tests exercise model transitions, size compatibility, service decisions, pickup-code rules, distance calculation, fake behavior and logging decorators. Mockito isolates repositories or ports only when their interaction is the subject; for example, ParcelServiceTest can verify that hardware failure prevents a state transition without needing PostgreSQL. These tests live under the model, service, fake, geocoding, port and config test packages and can be run individually with a command such as ./mvnw -Dtest=ParcelTest test from hw1/.

Integration tests exercise Spring MVC/controllers, services, JPA and PostgreSQL together, while using deterministic hardware, notification and geocoding adapters. They verify the HTTP mapping as well as persisted state and seed behavior; for example, RegisterParcelIntegrationTest checks the allocation result, status code and compartment row. The tests are mainly under controller, service and config, run with commands such as ./mvnw -Dtest=RegisterParcelIntegrationTest test, and require Docker because the PostgreSQL database is provided by Testcontainers. Dedicated concurrency integration tests coordinate operations against PostgreSQL to verify locks and lifecycle invariants, ./mvnw -Dtest=ApiConcurrencyIntegrationTest test is one focused command, while ParcelAllocationConcurrencyIntegrationTest targets the last-compartment race.

Contract tests send requests through MockMvc and validate serialized responses against JSON Schema so ./mvnw -Dtest=ApiContractTest test checks compatibility of representative API shapes. Acceptance tests use the eight Gherkin feature files under src/test/resources/features, Cucumber step definitions arrange scenario data, drive HTTP operations and assert business outcomes, and the suite runs with ./mvnw -Dtest=CucumberTest test. These are domain-readable acceptance examples, not browser automation.

The web layer has two complementary forms of verification. WebInterfaceUsabilityTest inspects HTML for labels, navigation, status feedback, semantic landmarks and responsive metadata without launching a browser; run it with ./mvnw -Dtest=WebInterfaceUsabilityTest test. PlaywrightEndToEndTest launches Chromium, navigates the real pages and submits user interactions against a random-port Spring application so after installing Chromium once, run it with ./mvnw -Dtest=PlaywrightEndToEndTest test.

Finally, non-functional evidence includes the JaCoCo line/branch gate, deterministic concurrency checks and the short local k6 mixed-load profile at performance/k6/tuttibox.js. The full Java suite and coverage gate run with ./mvnw clean verify meaning k6 runs separately with k6 run performance/k6/tuttibox.js against an isolated running app/database. The k6 profile measures the five static-page responses, station search, parcel registration/tracking and operator reads at a bounded load. It does not measure production

capacity, real external-provider latency or full browser rendering. Accessibility conformance remains partially unverified. These distinctions matter: a unit test proves a local rule, an integration test proves component interactions, a contract test protects response shape, a BDD scenario expresses acceptance behavior, a browser test checks the UI wiring, and a load test measures response behavior under one stated synthetic profile.

The last available local Surefire reports in target/surefire-reports record 273 tests across 38 report classes, with 0 failures, 0 errors and 0 skipped. JaCoCo's local XML/HTML report records 97.09% line coverage (701/722 lines) and 86.93% branch coverage (153/176 branches).

The test levels provide different kinds of evidence. A unit test can show that ParcelService refuses a pickup after its deadline with a fixed clock, but cannot show that Spring maps that failure to the documented HTTP status. An integration test can observe the controller, service, transaction, PostgreSQL row and fake hardware together, but does not prove that a browser form is wired to the right endpoint. A contract test checks the serialized response shape, but does not prove that the business rule leading to that response is correct. The Cucumber layer expresses acceptance examples in domain language, and Playwright adds the visible browser interaction. The overlapping checks are purposeful: each catches a different class of regression.

### 3.2 Unit tests and test doubles

Unit tests focus on pure rules and service decisions. ParcelTest covers the initial state, valid cancellation/drop-off/pickup/expiration/return sequences, invalid transitions and deadline boundaries. CompartmentTest covers FREE, RESERVED, OCCUPIED and OUT_OF_SERVICE transitions. CompartmentSizeTest checks the compatibility ordering and parsing. PickupCodeTest checks code format, normalization, deterministic hashing, matching and rejection of ambiguous/invalid formats. StationSearchServiceTest verifies Haversine distance ordering, tie breaking and availability counts.

ParcelServiceTest uses a fixed Clock and Mockito mocks for repositories, hardware and notifications so it can verify whether calls happen, whether failed operations preserve state and whether deadlines are respected without starting PostgreSQL. LoggingHardwarePortTest mocks only the delegate to assert that calls/results are logged and exceptions propagated. This is an intentional use of mocks: these tests examine interaction and service decisions, not persistence semantics. Database behavior is tested with PostgreSQL in integration tests instead of mocked repositories.

For example, testDropOffOpensCompartment arranges a CREATED parcel with a reserved assigned compartment, invokes the drop-off operation and verifies that the hardware port receives the station/compartment pair. It also checks the important negative assertion: opening the door alone does not mark the parcel ready. testConfirmDropOffWithClosedDoor then configures successful close confirmation and a closed-door sensor result; its assertions cover the state transition, occupied compartment, hashed code persistence and notification call. The paired testConfirmDropOffWithOpenDoor and testConfirmDropOffHardwareCloseFailure cases make sure those same side effects do not happen if the sensor says the door is still open or the hardware adapter throws. This arrange,

act and assert structure makes the tested invariant visible instead of asserting only that a method returned.

For time-dependent behavior, tests such as testDropOffAfterDeadlineRejected, testDropOffAtDeadlineAccepted, testPickupAfterDeadlineRejected and testPickupAtDeadlineAccepted position the injected clock immediately before, at or after the boundary. This documents the inclusive boundary policy (the deadline instant itself is still accepted) and avoids slow, flaky Thread.sleep tests. testApplyDeadlinesCancelsParcelAndReleasesCompartment contrasts with testApplyDeadlinesExpiresParcelAndKeepsCompartmentOccupied: a parcel that has not been deposited releases its reservation when its drop-off deadline passes, while a parcel that is already ready remains physically in the locker after the pickup deadline and requires operator return/removal.

StationSearchServiceTest.testStationsAreRankedByDistance gives the geocoder a known coordinate, supplies stations with known coordinates, and checks nearest-first ordering. Separate tests cover the mathematical properties of the Haversine helper (same point gives zero and reversing the points gives the same distance), tie ordering, empty stations, an unknown address and every geocoding failure reason. These assertions distinguish address resolution errors from a valid address for which there are simply no stations.

Three stateful fakes support repeatable tests and demo use:

- **InMemoryHardwareFake** tracks door state keyed by station and compartment; switches inject failure to open, failure to confirm closing, or a door that remains open after confirmation. Tests assert both the thrown error and parcel/compartment state.
- **InMemoryGeocodingFake** maps known addresses to coordinates, supports addresses added by a test, returns empty for unknown addresses and can inject each GeocodingException.Reason. Test properties select provider=memory, so the default test build never calls Nominatim.
- **InMemoryNotificationFake** stores emitted (recipientContact, trackingCode, pickupCode) records in memory and logs the code for the demo andit lets tests inspect notification contents.

The production/demo Spring configuration always wraps the hardware fake in LoggingHardwarePort. It is a working locker fake by default, satisfying the end-to-end terminal requirement without devices.

### 3.3 Integration tests

Integration suites use @SpringBootTest and Spring MVC test support to exercise actual controllers and service wiring. TestcontainersConfiguration starts a PostgreSQL container and registers it as the application datasource; test properties set Hibernate to create-drop, disable background scheduling and seeding, and select the memory geocoder. @AutoConfigureMockMvc allows HTTP-level assertions without binding a fixed port. This gives faster repeatable controller/API tests while still exercising PostgreSQL, transactions and JPA. Playwright uses a random server port to cover actual browser behavior.

Concrete integration examples include:

- **RegisterParcelIntegrationTest** checks the selected smallest compatible compartment, S parcel allocation, full station 409, unknown station 404 and malformed inputs.
- **CourierDropOffIntegrationTest, ParcelServiceIntegrationTest** and **RecipientPickUpIntegrationTest** inject hardware failures and verify that invalid physical outcomes do not advance state or release the locker.
- **ParcelDeadlineIntegrationTest** verifies cancellation/expiration using historical deadlines and repository state, with no sleep-based waiting.
- **OperatorSupervisionIntegrationTest** checks station/state filters, expired removal, compartment service state and invalid operations.
- **SeedDataIntegrationTest** confirms three stations/eighteen compartments and idempotent reapplication.
- **StationSearchControllerIntegrationTest** verifies unknown address and geocoder failure mappings. NominatimGeocodingAdapterTest starts a local JDK HttpServer to return successful, empty, rate-limited, server-error, malformed and delayed responses meaning the provider is not contacted over the network.

The registration suite demonstrates the difference between allocation policy and HTTP contract. testRegisterParcelPicksSmallestFit seeds free compartments of multiple sizes, sends an M request and checks that the medium compartment is assigned while the larger one remains free. testRegisterParcelNoCompartmentAvailable observes 409 and the absence of an extra parcel, testRegisterParcelUnknownStation observes 404. The test therefore verifies both the body/status seen by a partner and the durable state behind it. In the courier suite, a successful sequence is observed at each boundary: after the first drop-off request the state remains CREATED and after close confirmation it is READY_FOR_PICKUP, the compartment is occupied and notification fake holds the issued code. A wrong compartment returns 400, while an open door or injected close failure returns 503 and leaves the parcel CREATED, the pickup hash absent and the compartment reserved.

### 3.4 Functional tests (web automation)

PlaywrightEndToEndTest launches Chromium once for the suite, starts the Spring application on a random port and creates a fresh browser context for each test. Before each test it deletes parcels, compartments and stations and resets the geocoding, hardware and notification fakes. This prevents cookies or prior scenario records from influencing another test. Locators use visible roles or stable input IDs, and Playwright waits for rendered result text rather than relying on fixed sleeps.

The method allMainPagesRenderInChromium visits the search, courier, recipient, tracking and operator pages and checks their main landmark, heading, footer and stylesheet link. stationSearchRendersLiveResults adds an address to the memory geocoder, fills the #address field and activates the search button through its accessible role, it then waits for the station name and asserts the rendered row/capacity. parcelLifecycleWorksAcrossBrowserTerminals begins with a real database reservation, fills the courier form using associated labels, opens and confirms the compartment, and checks the rendered READY_FOR_PICKUP state. It then follows the tracking page and history, obtains the code from the in-memory demo notification record, submits it in the recipient page and checks PICKED_UP plus released capacity. Finally, the operator test exercises both out-of-service and restore actions and verifies the visible compartment state. This route

is deliberately cross-page: it catches mismatches among HTML IDs/labels, JavaScript request construction, endpoint routes, JSON properties and backend state that isolated API tests cannot detect.

The four browser tests cover (1) rendering all five pages with their main landmark, heading, footer and shared stylesheet, (2) station search returning a seeded station and availability row, (3) courier deposit followed by browser tracking and recipient pickup, and (4) operator taking a free compartment out of service and restoring it. These journeys were selected because they exercise the principal visible workflows and bridge browser requests to persisted state. They use the real browser but deterministic fake geocoding/hardware/notifications, so they do not depend on Nominatim, SMS or a physical locker.

The setup and teardown are part of the browser-test reliability story. Each test gets a new browser context (no shared cookies/local storage), while the @BeforeEach reset removes persisted records and resets the stateful fakes. The browser itself is shared at suite scope to avoid paying the startup cost for every method. The application uses a random port so another local process on port 8080 cannot make the test accidentally talk to a developer's running instance. This reduces cross-test interference, although the tests still require Docker for PostgreSQL and an installed Playwright Chromium binary.

WebInterfaceUsabilityTest is a separate MockMvc-based HTML inspection suite. It checks page language/charset/viewport, common layout and navigation, labels and input hints, table headings, live status regions, reset actions and multi-step courier feedback.

### 3.5 Acceptance tests (BDD)

The eight Cucumber feature files under src/test/resources/features/ organize examples by UC1–UC8: station search, parcel registration, courier drop-off, recipient pickup, tracking and lifecycle, operator supervision, deadlines, and web terminals/seed data. Together, the current Surefire output reports 60 Cucumber scenarios. The Gherkin examples include success paths and relevant rejection/failure cases, including invalid codes, wrong compartments, injected hardware faults, deadlines, full stations, concurrency and out-of-service lockers.

Step definitions drive Spring MockMvc, not the browser. ApiSupport stores each response in scenario-scoped ScenarioState, setup steps create stations, compartments and parcels in the database, and assertion steps inspect HTTP status/JSON and persisted/fake hardware state. This level makes business acceptance examples fast and deterministic while Playwright separately covers browser wiring. Time scenarios manipulate deadlines and directly invoke deadline processing instead of waiting for the real scheduler. Hardware-failure scenarios configure InMemoryHardwareFake and inspect that the operation did not complete falsely.

Representative scenarios show how the prose requirements become executable examples. In UC2, “Registering M selects the smallest compatible compartment” checks the assignment, while “Concurrent registrations do not double-book the last compartment” starts competing registrations and checks the final single reservation. In UC3, “Courier opens assigned compartment and door confirmation marks parcel ready” contrasts with “Closing failure does not mark parcel ready”: a status transition is accepted only after the simulated physical precondition. In UC4, “Pickup code can only be used once” runs a successful pickup and retries the same code; the second request must not reopen the door or reclaim the

parcel. UC5 covers tracking history and cancellation rules; UC6 includes “Occupied compartment cannot be disabled” and UC7 distinguishes expiration before deposit, which releases a reservation, from pickup expiration, which leaves a parcel physically present. UC8 checks that the terminal pages are served and seed initialization can be repeated. The scenario names make the requirement links auditable, while the step code defines the exact fixtures and assertions.

### 3.6 Non-functional testing

- **Concurrency:** Concurrency testing answers a different question from load testing: it checks that simultaneous requests preserve business invariants, even if the overall request rate is low. The key invariant in TuttiBox is that two partners cannot reserve the same final compartment. ParcelAllocationConcurrencyIntegrationTest starts two executor tasks, waits until both are ready on a CountDownLatch, then releases them together against one S compartment in a PostgreSQL-backed Spring context. The expected outcome is exactly one successful registration and one NoAvailableCompartmentException, the final database must contain one parcel and show the compartment as RESERVED. This tests the repository lock/transaction behavior and durable outcome, not throughput. Other concurrency integration suites exercise API-level races in parcel lifecycle operations, including pickup and competing state changes. Their oracles check response/result cardinality, final parcel state, pickup-code single-use behavior and compartment status. A passing race test means the tested interleaving preserved those invariants and it does not prove that every possible thread schedule, database failover or distributed deployment is safe. The database lock is local to PostgreSQL, and the current system has no multi-region or distributed-lock design. The k6 lifecycle scenario adds concurrent request pressure at the service boundary. In the recorded run, all 965 registration flows completed and were cancelled, all associated status/history checks passed, and no request returned an unexpected status. This is useful evidence that the exercised transactions and read paths remained responsive under that small mixed workload. For stronger concurrency evidence, a future test should add an explicit synchronized k6 burst against a known station with one remaining compartment, assert one 201 and the remaining 409 responses, then cancel the successful parcel and verify the database inventory. That scenario should run on a fresh isolated database because earlier test data or concurrent runs can change available capacity.
- **Performance:** The project now includes a reproducible k6 workload at hw1/performance/k6/tuttibox.js. It covers representative static-page, station-search, parcel and operator-read paths without depending on third-party availability. The station-search scenario looks up Aveiro, verifies HTTP 200 and the three seeded stations, and ramps from one to five iterations per second. The web-page scenario requests all five HTML pages and checks their status and content type, with an arrival rate peaking at two requests per second. The parcel scenario models partner registration followed by tracking and operator reads: each virtual user registers an S parcel, reads its status, history and details, queries the station's parcel filter and compartment inventory, then cancels the parcel so its reservation returns to the free pool. It ramps from one to six virtual users. This k6 profile currently runs locally and is not a step in .github/workflows/ci.yml. The measured run lasted 55 seconds.. The application selected TUTTIBOX_GEOCODING_PROVIDER=memory, hardware and notification boundaries used their in-memory demonstration adapters. The host CPU and memory allocation were not recorded, so the measurements are local comparative evidence rather than a portable benchmark.

- **Accessibility:** Current automated checks cover useful foundations: lang="pt", UTF-8, responsive viewport, semantic main/header/footer, explicit labels, table scopes, role="status", aria-live="polite", visible status text and consistent cross-page navigation. Playwright confirms that the primary page content is visible in Chromium.

### 3.7 Code quality analysis

- **JaCoCo:** jacoco-maven-plugin 0.8.15 attaches its agent during Maven build, generates HTML/XML at verify, and enforces bundle minimums of 90% line and 80% branch coverage. The last available report shows 701 of 722 executable lines covered (97.09%) and 153 of 176 branches covered (86.93%) meaning both pass the configured gate. Coverage indicates which instrumented lines/branches ran, not whether assertions are sufficient or requirements are complete.
- **SonarQube:** The Maven project declares SonarScanner for Maven, project key tuttibox-hw1, local server URL http://localhost:9000 and JaCoCo XML path. This tool was only introduced in the course's practical sessions very recently jjust last week. Consequently, when I incorporated it into the project, it served primarily as an aid for debugging and, more importantly, for identifying issues I hadn't previously noticed such as duplicate code given that the business logic implementation was already nearly complete. Nevertheless, even based on this limited use over the past week, I am certain that it is an extremely valuable tool for monitoring development. I believe it proved significant for detecting bugs and vulnerabilities during the final stretch of the project.

### 3.8 Continuous integration pipeline

.github/workflows/ci.yml runs on push and manual dispatch. It checks out the project, sets up Temurin Java 21 with Maven dependency caching, installs Playwright Chromium with system dependencies, then executes ./mvnw -B -ntp clean verify from hw1/. That command compiles the application, executes Surefire-discovered JUnit/Cucumber/contract/integration/usability/E2E tests, generates JaCoCo reports and fails the build if the line/branch gate is missed. The workflow publishes JUnit summaries and uploads Surefire and JaCoCo artifacts even after a failed run where artifacts exist.

The full test suite and JaCoCo gate are therefore enforced in CI. The SonarQube server is not provisioned in CI and no SONAR_TOKEN-conditioned analysis step exists, Sonar analysis remains local and separate.

## 4 References & resources

**Project resources**

| Resource | URL/location |
|---|---|
| Git repository | https://github.com/InesLBatista/TuttiBox-Parcel-System |
| Video demo | *docs/demo.mp4* |
| API documentation | http://localhost:8080/swagger-ui/index.html |
| CI/CD pipeline | https://github.com/InesLBatista/TuttiBox-Parcel-System/actions/workflows/ci.yml |

**Reference materials**

The selected provider is the public Nominatim service at https://nominatim.openstreetmap.org, specifically the Search API endpoint /search. The adapter submits a free-form address query as q, asks for format=jsonv2, sets limit=1, restricts to Portugal with countrycodes=pt, requests Portuguese results with accept-language=pt, and identifies the project using the configured User-Agent TuttiBox-HW1/1.0 (TQS academic project). The first result's lat and lon are parsed into coordinates, the service calculates distance to seeded stations itself using the Haversine formula and sorts by distance, then station code.

The data flow separates geocoding from locker inventory. For a request such as GET /api/stations/search?address=Universidade%20de%20Aveiro, NominatimGeocodingAdapter resolves the text address to one origin coordinate. StationSearchService then calls StationRepository.findAll() to load the TuttiBox stations persisted by SeedData or later maintained in the database, calculates the Haversine distance from the origin to each station, and obtains free capacity through CompartmentRepository for compartments with status FREE. It sorts those station/capacity results and returns NearbyStationDto objects. Thus, Nominatim supplies the search origin, the application's own database supplies which lockers exist and whether each size is available. The seed is not a substitute for calling Nominatim in the normal nominatim configuration, and Nominatim is not a source of seed records.

The adapter uses a five-second connect/request timeout and maps provider behavior into explicit domain reasons: timeout, unavailable connection, HTTP 429 rate limit, provider HTTP errors, invalid response and no result. A no-result response is an empty optional and becomes 200, failures map through the global handler to 504, 503 or 502. Tests deliberately replace only the geocoding dependency: NominatimGeocodingAdapterTest uses a local HTTP server to exercise the external API's request/response and error handling, while service and end-to-end tests use the memory geocoder for stable coordinates. Both kinds of test still use known TuttiBox station fixtures or seeded database rows so they can verify ranking and capacity. No test calls the public Nominatim server, and using seed data does not mean the live adapter is bypassed in a normal run; the TUTTIBOX_GEOCODING_PROVIDER=memory setting explicitly selects the fake when deterministic/offline behavior is wanted.

Nominatim's public-service policy requires a valid identifying User-Agent or Referer, no more than one request per second per application, appropriate OpenStreetMap attribution, prohibits API-backed autocomplete and asks clients not to submit confidential information. The current User-Agent is compliant and the address suggestions in the UI are hard-coded local examples, not an autocomplete API. The application still needs a global throttle and visible attribution, limit=1 is only the number of results returned and does not implement the one-request-per-second rule. Nominatim also recommends caching/proxying where possible.